

Snake 3D: Implementação de um Jogo Clássico com OpenGL/GLUT e Técnicas de Computação Gráfica

Antônio Enzo Ferreira do Nascimento

¹Ciência da Computação – UFPI
Teresina – Piauí – Brasil

antonio.do@ufpi.edu.br

Abstract. *This paper presents the development of a three-dimensional version of the classic Snake game, implemented in C++ using the OpenGL fixed-function pipeline and the GLUT toolkit. The application features discrete grid-based movement, progressive difficulty, a randomly oriented central wall that subdivides the arena, teleportation portals, three camera modes, Gouraud shading, real textures loaded from image files, 3D models parsed from OBJ files, a skybox, and audio effects. The goal is to apply core Computer Graphics concepts, geometry, illumination, texturing, and animation in a complete, interactive, real-time application built from scratch.*

Resumo. *Este artigo apresenta o desenvolvimento de uma versão tridimensional do clássico jogo Snake, implementada em C++ com o pipeline fixed-function do OpenGL e o toolkit GLUT. A aplicação incorpora movimentação discreta em grid, dificuldade progressiva, parede central sorteada (vertical ou horizontal) com portais de teleporte, três modos de câmera, sombreado de Gouraud, texturas reais, modelos 3D em formato OBJ, skybox e efeitos sonoros. O objetivo é aplicar os pilares de Computação Gráfica, geometria, iluminação, texturização e animação em uma aplicação interativa completa.*

1. Introdução

O Snake é um dos jogos mais icônicos da história dos videogames. Criado em 1976 por Gremlin Industries na forma de *Blockade* e popularizado nos celulares Nokia no final da década de 1990, o jogo desafia o jogador a mover uma cobra por um campo fechado, coletar itens que a fazem crescer e evitar colidir consigo mesma ou com as bordas do mapa [Wolf 2012].

Este trabalho propõe uma **reimplementação tridimensional do Snake** usando a API **OpenGL** com o pipeline *fixed-function* e o toolkit **GLUT**, como trabalho final da disciplina de Computação Gráfica. O objetivo é ir além de um exercício bidimensional e construir uma aplicação interativa completa que integre: renderização 3D com iluminação e texturas, movimentação temporal controlada por timer, mecânicas de jogo progressivas e efeitos visuais animados.

Regras do jogo

A cobra se move em passos discretos sobre um *grid* de 30×30 células. A cada passo ela avança uma célula na direção atual; o jogador pode alterar a direção com as teclas WASD ou pelas setas do teclado. Ao comer uma maçã, a cobra cresce um segmento e

a velocidade aumenta. A partida termina se a cabeça atingir a borda do mapa, a parede central ou o próprio corpo da cobra.

Com 5 maçãs consumidas, uma parede de pedra divide a arena ao meio, com orientação (vertical ou horizontal) sorteada aleatoriamente. Simultaneamente, dois **portais de teleporte** aparecem, um de cada lado da parede, permitindo à cobra atravessar o obstáculo.

Objetivos técnicos

- Representar a cena em 3D com câmeras comutáveis (terceira pessoa, primeira pessoa e vista de topo);
- Aplicar iluminação de Gouraud com uma fonte de luz direcional;
- Carregar e exibir texturas reais (JPG/PNG) e modelos 3D (OBJ);
- Implementar efeitos visuais animados (portais, maçã flutuante, *skybox*);
- Reproduzir efeitos sonoros e música de fundo de forma multiplataforma.

2. Materiais

O projeto foi desenvolvido em C++ (padrão C++11) e utiliza exclusivamente bibliotecas disponíveis no ambiente acadêmico da disciplina, sem dependências externas adicionais. A Tabela 1 lista as ferramentas e bibliotecas utilizadas.

Tabela 1. Ferramentas e bibliotecas utilizadas no projeto

Biblioteca / Ferramenta	Finalidade
OpenGL (pipeline <i>fixed-function</i> , 1.x/2.1)	Renderização 3D: geometria, texturas, <i>blending</i> , iluminação via <code>glLight*</code> e <code>glColorMaterial</code> .
GLU (<i>OpenGL Utility Library</i>)	Projeção (<code>gluPerspective</code>), câmera (<code>gluLookAt</code>), <i>quadrics</i> (<code>gluSphere</code> , <code>gluCylinder</code>) e <i>mipmaps</i> (<code>gluBuild2DMipmaps</code>).
GLUT / <code>freeglut</code>	Janela, loop de eventos, <i>callbacks</i> de teclado e mouse, timer, primitivas prontas (<code>glutSolidSphere</code> , <code>glutSolidTorus</code>) e texto bitmap do HUD.
<code>stb_image.h</code> [Barrett 2024]	Carregamento de imagens JPG/PNG para texturas (biblioteca de cabeçalho único em domínio público, incluída diretamente no projeto).
<code>winmm</code> (Windows)	Reprodução de efeitos sonoros WAV (<code>PlaySoundA</code>) e BGM MP3 (<code>mcisendstringA</code>) no Windows.
<code>afplay</code> (macOS)	Reprodução de áudio via ferramenta nativa do sistema no macOS (<code>fork + execvp</code>).
<code>g++</code> / <code>Falcon C++</code>	Compilação do projeto no ambiente Windows com a flag <code>-DGLUT_STATIC</code> .

Os *assets* visuais incluem texturas CC0 do **ambientCG** [ambientCG 2024] (grama e pavimento), escamas de cobra do **3DTextures.me**, o modelo 3D da maçã do **Poly Haven** [Poly Haven 2024] e o *skybox* de **Emil Persson (Humus)** [Persson 2012], licenciado sob CC BY 3.0.

3. Métodos

3.1. Representação do *grid* e movimentação discreta

A arena é um *grid* de **30×30 células** (1 unidade GL cada). Posições são inteiras, convertidas para o mundo por funções inline (*gx*, *gz*). A cobra é `vector<Cell> snake` (índice 0 = cabeça); o estado de cada célula fica em `gridState[30][30]` (CT_EMPTY ou CT_WALL).

O movimento é **discreto e temporal**: `step()` avança uma célula e é re-agendada pelo timer GLUT a cada `stepMs` ms (150 ms inicial, mínimo 60 ms, reduzindo 5 ms por maçã). Duas variáveis `curDir/nextDir` evitam inversões instantâneas.

3.2. Colisões e condições de fim de jogo

Três condições encerram a partida: (1) **saída do *grid*** (`!cellInGrid`); (2) **colisão com CT_WALL**, cobre a **borda sólida** permanente (anel em índices 0 e GRID-1) e a parede central dinâmica; (3) **auto-colisão**. Em qualquer caso, `gameOver = true` e o som de *game over* é reproduzido.

3.3. Parede central e portais

Com 5 maçãs, `spawnWall()` sorteia a orientação 50/50 (**vertical**: coluna GRID/2; **horizontal**: linha GRID/2) e chama `spawnPortals()`, que posiciona um portal de cada lado em célula livre. Parede vertical: $x \in [1, \text{WALL_COLUMN})$ e $x \in (\text{WALL_COLUMN}, \text{GRID} - 2]$; horizontal: análogo em z . Ao entrar num portal (`checkPortal`), a cobra é teleportada para a célula seguinte ao portal de saída; um **cooldown de 3 passos** evita re-entrada imediata.

3.4. Câmeras

A tecla C alterna três modos (todos via `gluLookAt + gluPerspective`): **3ª pessoa** orbital controlada pelo mouse (`camDist = 28`); **1ª pessoa** na cabeça da cobra, FOV 75°; **Topo**, câmera zenital com `up = (0, 0, -1)`.

3.5. Iluminação e sombreamento

O jogo usa **sombreamento de Gouraud** (GL_SMOOTH) com pipeline *fixed-function* sem GLSL. A única fonte (GL_LIGHT0) é direcional (`w=0`), com componente ambiente 0,35 (sombras legíveis), difusa levemente quente (0,90/0,88/0,80) e especular 0,30.

GL_COLOR_MATERIAL vincula `glColor3f` ao material difuso; GL_NORMALIZE recomputa normais após escala; GL_LIGHT_MODEL_TWO_SIDE evita faces pretas em malhas OBJ com normais inconsistentes. *Skybox*, portais e HUD desativam a iluminação para preservar cores puras.

3.6. Texturas e modelos 3D

Texturas são carregadas via `stb_image.h` [Barrett 2024] com *fallback* procedural 64×64 caso o arquivo falte; o skybox usa `GL_CLAMP_TO_EDGE` para evitar costuras; texturas com atlas (maçã) desabilitam mipmap para evitar vazamento do fundo preto.

O **loader OBJ próprio** (`loadOBJ`) faz *parse* de `v/vt/vn/f` nos quatro formatos, normaliza o modelo por *bounding box* e compila o resultado em uma *display list* (`glCallList`). O corpo da cobra usa *quadrics* GLU: esferas nas juntas e cilindros com raio interpolado pelo pescoço à cauda (`bodyRadiusAt`).

3.7. Efeitos visuais

Os **portais** combinam núcleo pulsante, esfera *wireframe*, dois torus e 24 partículas orbitando, todos animados por `GLUT_ELAPSED_TIME` com *blending* aditivo (`GL_SRC_ALPHA`, `GL_ONE`) e `glDepthMask(GL_FALSE)`. O **skybox** é um cubo de 400 unidades centrado no olho, desenhado sem *depth test*. A **maçã** flutua e gira por seno do tempo. O **HUD** usa `gluOrtho2D` exibindo pontuação, câmera ativa e painel de *game over*.

4. Resultados e Discussão

O jogo foi compilado e executado com sucesso com **Falcon C++** e `glut32`. A taxa lógica varia de **6,7 a 16,7 passos/s** ($150 \rightarrow 60$ ms/passos), todos os sistemas funcionam conforme especificado e a portabilidade é garantida pelos *fallbacks* procedurais e pela compilação condicional de áudio (`#ifdef _WIN32 / _APPLE_`). As figuras abaixo ilustram o jogo em execução.

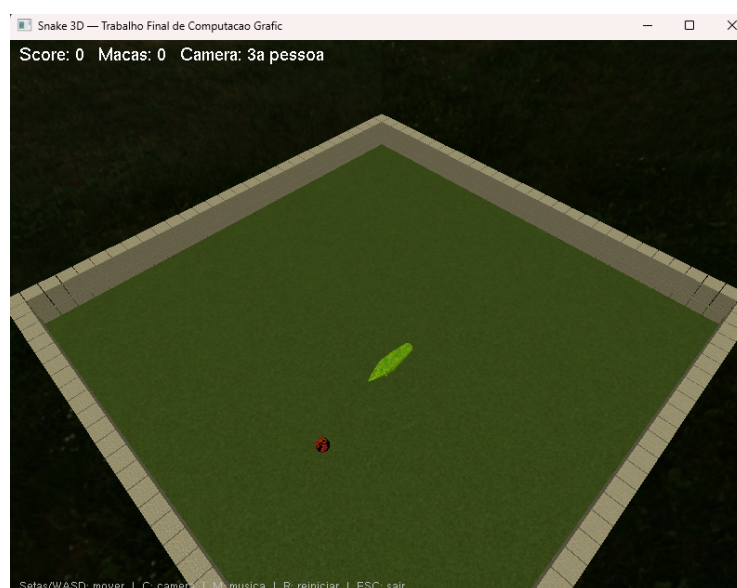


Figura 1. Câmera em terceira pessoa: cobra com corpo afinado, maçã 3D flutuante e skybox. A borda sólida do mapa é visível ao redor do *grid*.



Figura 2. Parede central (textura de *paving stones*) com os dois portais animados em evidência. Portal A (laranja) e Portal B (azul) ocupam lados opostos da parede.

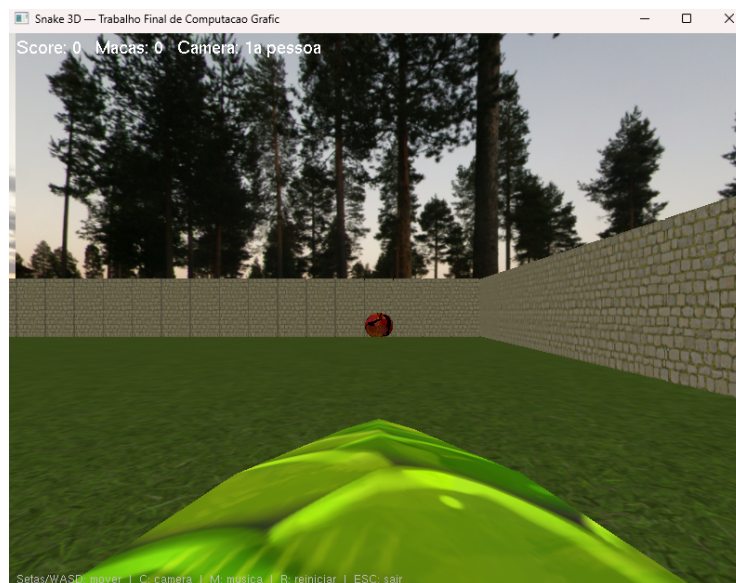


Figura 3. Câmera em primeira pessoa: visão da perspectiva da cabeça da cobra, evidenciando o sombreado de Gouraud no chão e nas paredes.

5. Conclusão

O Snake 3D demonstrou a integração prática dos pilares de Computação Gráfica em uma aplicação interativa. Os principais desafios foram: (1) **loader OBJ próprio** com suporte aos quatro formatos de face e normalização por *bounding box*; (2) **orientação dos cilindros** via $\text{atan2} + \text{glRotatef}$ em Y; (3) **atlas da maçã sem mipmap**, fundo preto vazava para as bordas, resolvido com `GL_LINEAR` sem mipmap; (4) **normais invertidas** em modelos OBJ eliminadas com `GL_LIGHT_MODEL_TWO_SIDE`; (5) **BGM multiplataforma** (`mcisendstringA` no Windows vs. `fork+afplay` no macOS).

Como trabalhos futuros destacam-se: *shaders* GLSL (Phong, *normal mapping*, sombras), múltiplas fases, *high score* persistente e IA.

Referências

ambientCG (2024). ambientCG — free pbr materials. Texturas de grama (Grass005) e pavimento (PavingStones138), licença CC0. Acesso em: jun. 2025.

Barrett, S. T. (2024). stb_image.h: Image loading/decoding library. Biblioteca de cabeçalho único em domínio público. Acesso em: jun. 2025.

Persson, E. (2012). Humus — 3d graphics resources. Skybox utilizado no projeto, licença CC BY 3.0. Acesso em: jun. 2025.

Poly Haven (2024). Poly Haven — free hdri, textures and 3d models. Modelo 3D da maçã (food_apple_01), licença CC0. Acesso em: jun. 2025.

Wolf, M. J. P. (2012). *Encyclopedia of Video Games: The Culture, Technology, and Art of Gaming*. Greenwood, Santa Barbara, CA.